

```
import numpy as np
import pandas as pd
```

```
Carseats = pd.read_csv("Carseats.csv")
```

```
Carseats
```

	Sal es	CompPr ice	Inco me	Advertisi ng	Populati on	Pric e	ShelveL oc	Ag e	Educati on	Urb an	U S
0	9.50	138	73	11	276	120	Bad	42	17	Yes	Ye s
1	11.2 2	111	48	16	260	83	Good	65	10	Yes	Ye s
2	10.0 6	113	35	10	269	80	Mediu m	59	12	Yes	Ye s
3	7.40	117	100	4	466	97	Mediu m	55	14	Yes	Ye s
4	4.15	141	64	3	340	128	Bad	38	13	Yes	N o
...	...	...	...	...	...	...	...	...	...	...	...
39 5	12.5 7	138	108	17	203	128	Good	33	14	Yes	Ye s
39 6	6.14	139	23	3	37	120	Mediu m	55	11	No	Ye s
39 7	7.41	162	26	12	368	159	Mediu m	40	18	Yes	Ye s
39 8	5.94	100	79	7	284	95	Bad	50	12	Yes	Ye s

39	9.71	134	37	0	27	120	Good	49
9								

```

from sklearn.model_selection import train_test_split
Carseats_encoded = pd.get_dummies(Carseats, drop_first=True)

X = Carseats_encoded.drop(columns=['Sales'])
y = Carseats_encoded['Sales']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

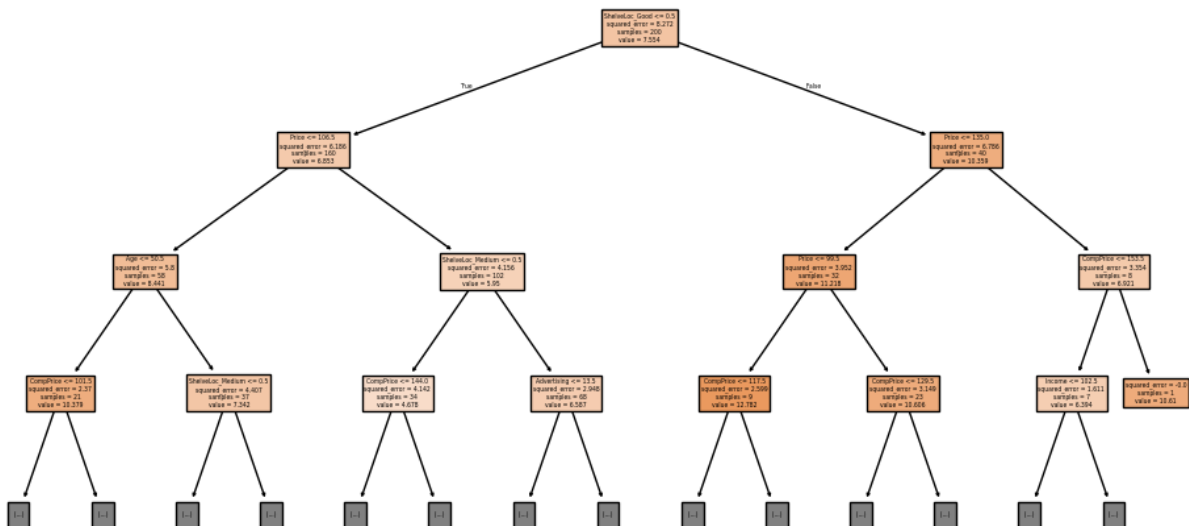
from sklearn.tree import DecisionTreeRegressor, plot_tree
from sklearn.metrics import mean_squared_error
import matplotlib.pyplot as plt

tree_model = DecisionTreeRegressor(random_state=42)
tree_model.fit(X_train, y_train)

# Plot the tree
plt.figure(figsize=(16, 10))
plot_tree(tree_model, feature_names=X.columns, filled=True)
plt.title("Regression Tree for Sales")
plt.show()

# Test MSE
y_pred = tree_model.predict(X_test)
test_mse = mean_squared_error(y_test, y_pred)
print("Test MSE (unpruned tree):", test_mse)

```



```

path = tree.cost_complexity_pruning_path(X_train, y_train)
ccp_alphas = path.ccp_alphas

kf = KFold(n_splits=10, shuffle=True, random_state=1)

cv_mse = []
for alpha in ccp_alphas:
    t = DecisionTreeRegressor(random_state=1, ccp_alpha=alpha)
    scores = cross_val_score(
        t, X_train, y_train,
        cv=kf, scoring="neg_mean_squared_error"
    )
    cv_mse.append(-scores.mean())

best_alpha = ccp_alphas[np.argmin(cv_mse)]
print("Best ccp_alpha:", best_alpha)

pruned_tree = DecisionTreeRegressor(random_state=1, ccp_alpha=best_alpha)
pruned_tree.fit(X_train, y_train)
y_pred_pruned = pruned_tree.predict(X_test)
mse_pruned = mean_squared_error(y_test, y_pred_pruned)
print("Pruned tree test MSE:", mse_pruned)
Best ccp_alpha: 0.3152323885135033
Pruned tree test MSE: 5.105824732518417

(d) Use the bagging approach in order to analyze this data. What test MSE do you obtain? Use the
feature_importance_ values to determine which variables are most important.

bag = BaggingRegressor(
    estimator=DecisionTreeRegressor(random_state=1),
    n_estimators=500,
    bootstrap=True,
    max_features=1.0,
    random_state=1
)

bag.fit(X_train, y_train)

y_pred_bag = bag.predict(X_test)
mse_bag = mean_squared_error(y_test, y_pred_bag)
print("Bagging test MSE:", mse_bag)
Bagging test MSE: 2.852212935372007
fi_bag = np.mean([est.feature_importances_ for est in bag.estimators_],
axis=0)
bag_importance = sorted(zip(X.columns, fi_bag), key=lambda x: x[1],
reverse=True)
print("Bagging variable importance (top):")
for name, val in bag_importance[:8]:
    print(f"{name}: {val:.3f}")
Bagging variable importance (top):
Price: 0.298
ShelveLoc_Good: 0.231
Age: 0.107

```

```
CompPrice: 0.084
ShelveLoc_Medium: 0.084
Advertising: 0.058
Income: 0.052
Population: 0.046
```

(e) Use random forests to analyze this data. What test MSE do you obtain? Use the `feature_importance_` values to determine which variables are most important. Describe the effect of `m`, the number of variables considered at each split, on the error rate obtained.

```
m_values = [2, 4, 6, 11] # 11 = all features = bagging-like
rf_results = {}

for m in m_values:
    rf = RandomForestRegressor(
        n_estimators=500,
        max_features=m,
        random_state=1
    )
    rf.fit(X_train, y_train)
    y_pred_rf = rf.predict(X_test)
    mse_rf = mean_squared_error(y_test, y_pred_rf)
    rf_results[m] = (mse_rf, rf.feature_importances_)

    print(f"Random forest (m={m}) test MSE: {mse_rf:.3f}")
    fi = sorted(zip(X.columns, rf.feature_importances_),
                key=lambda x: x[1], reverse=True)
    print(" Top variables:", [name for name, _ in fi[:5]])

Random forest (m=2) test MSE: 3.344
    Top variables: ['Price', 'ShelveLoc_Good', 'Age', 'CompPrice', 'Income']
Random forest (m=4) test MSE: 2.804
    Top variables: ['Price', 'ShelveLoc_Good', 'Age', 'CompPrice', 'Advertising']
Random forest (m=6) test MSE: 2.713
    Top variables: ['Price', 'ShelveLoc_Good', 'Age', 'CompPrice', 'Advertising']
Random forest (m=11) test MSE: 2.834
    Top variables: ['Price', 'ShelveLoc_Good', 'Age', 'CompPrice', 'ShelveLoc_Medium']
```